

Introduction to OS

Background material

- Check course webpage frequently at https://faculty.iitd.ac.in/~voroni/teaching/teaching_cse231/
- Contains details of textbooks and background material needed to follow the course
- Every class will have some exercises that would be given to ensure students become familiar with systems programming (including today)

Overall View of Course Content & References

- 1) Assembly Programming → Roy Sargforth's book
- 2) Concepts of OS → Operating Systems: Three Easy Pieces
- 3) Utilization of OS interfaces (also known as system calls) → The Linux Programming Interface by Michael Kerrisk
- 4) Programming/Modifying the Linux Kernel → Linux Kernel Programming by Robert Love

What is an OS?

Concept of abstractions: → We tend to understand complex systems using simplified models called abstractions.

For example, a large number of transistors together make an integrated circuit. A single CPU/motherboard's behavior is studied as a single unit and not as a collection of transistors.

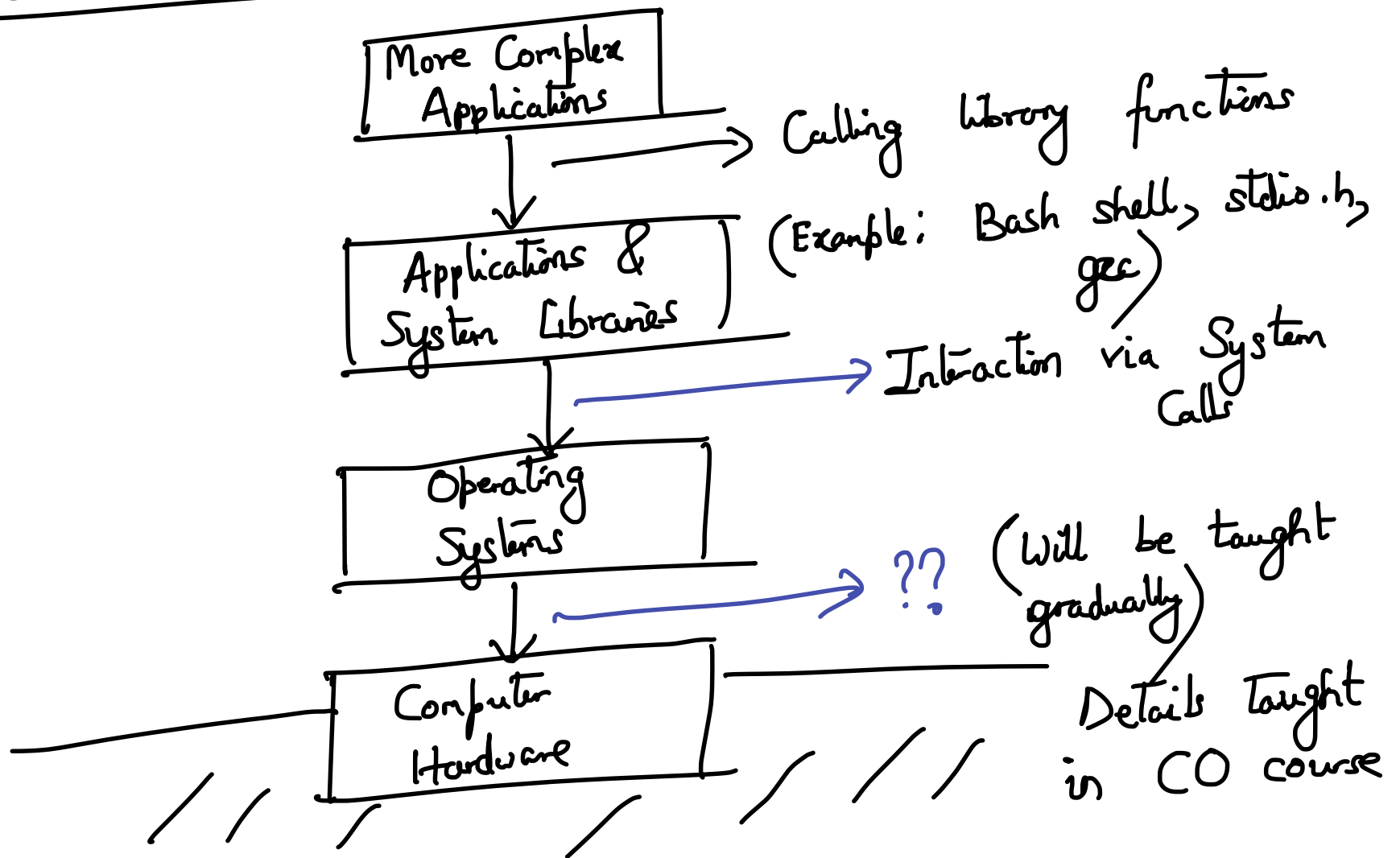
Abstracting the hardware: → The OS creates abstractions for the hardware for the benefit of application developers. This is to benefit the application programmers.

Disadvantage of abstractions: → 1) Leads to loss of performance, since abstractions add more layers of interaction.

2) Abstractions reduce the flexibility of interactions between different components of a system.

The first computers of the 1940's came with no OS, but soon programming became too complicated to be done without an OS.

Abstractions Used: →



Why Learn Assembly?

- 1) Close to the hardware, so some things like interaction with the hardware can only be done in assembly.
 - 2) Teaches more about the working of hardware.
- Actual interaction with hardware cannot be done in C language.

x86-64 Instruction Set Architecture (ISA)

Personal computers today mostly use x86-64 ISA.

The philosophy of x86-64 is that instructions should be complex enough to allow easy programming.

Currently, all Intel and AMD CPU's use
x86-64 ISA.

How did it evolve?

8085 μ processors $\rightarrow$ 8-bit
Can only operate on 8-bit integers.

8086 μ processors $\rightarrow$ 16-bit

x86 (Intel Pentium) μ processors $\rightarrow$ 32-bit

x86-64 (Today's desktops) $\rightarrow$ 64-bit

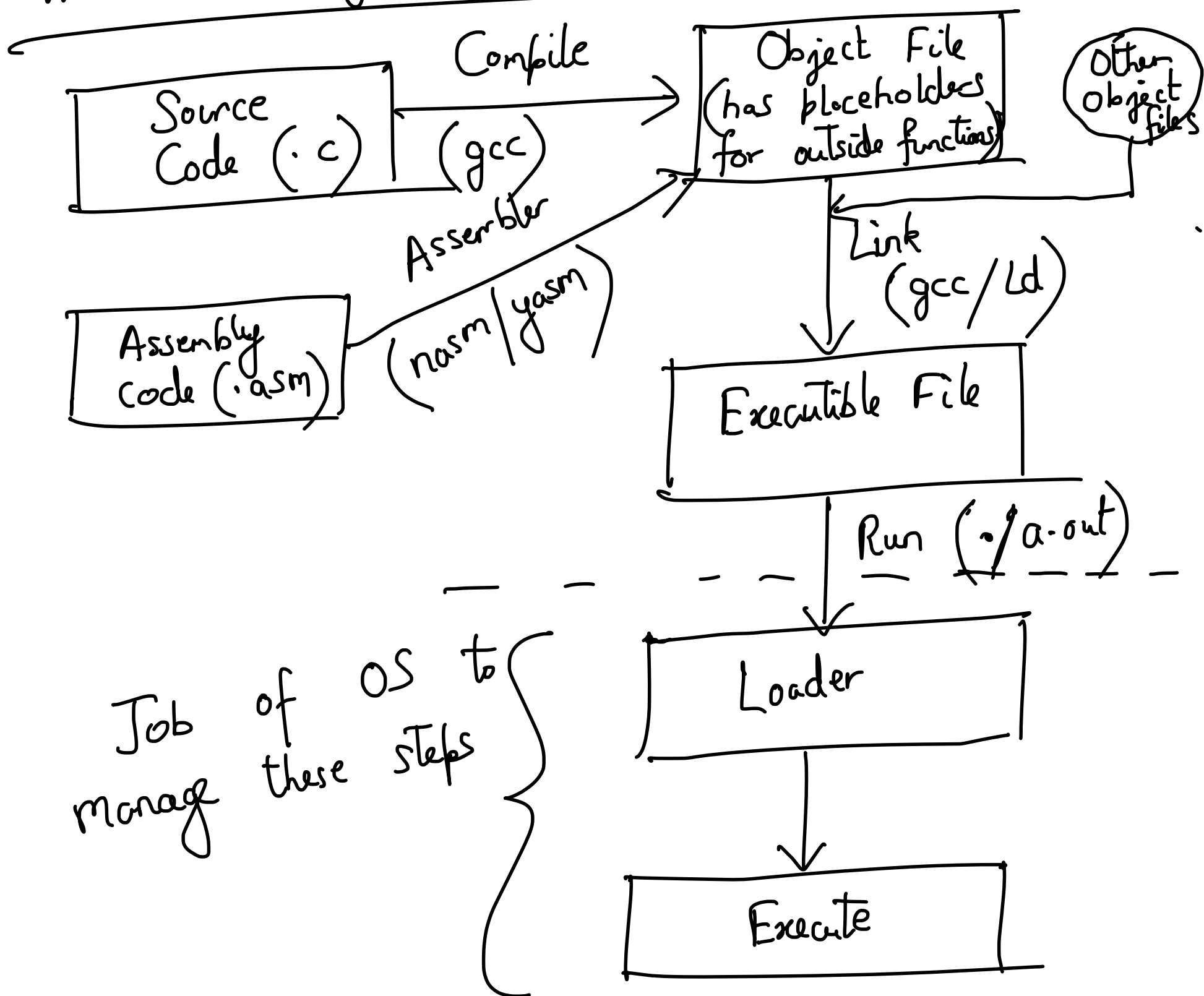
The key reason behind the success of these μ processors is backward compatibility, i.e. a new hardware/ μ processor can run old software.

Example: $\rightarrow$ Today's hardware can still run MS-DOS, which was for 16-bit machines.

This was not true for traditional microprocessors, such as IBM's PowerPC.

Disadvantages of backward Compatibility → There is a performance or cost penalty, since some silicon space must be assigned to handle both sets of instructions.

How do Programs Work?



In this course, we will use the yasm assembler because of its compatibility with the debugger.